

1. Write a function to calculate the factorial of a number using recursion.

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

2. Write a function to generate the nth Fibonacci number using recursion

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)
```

3. Write a function to find the sum of elements in a list using recursion.

```
def sum_list(arr):  
    if not arr:  
        return 0  
    else:  
        return arr[0] + sum_list(arr[1:])
```

4. Write a function to calculate the power of a number using recursion.

```
def power(base, exp):  
    if exp == 0:  
        return 1  
    else:  
        return base * power(base, exp - 1)
```

5. Write a function to check if a string is a palindrome using recursion.

```
def is_palindrome(s):
    if len(s) <= 1:
        return True
    else:
        return s[0] == s[-1] and is_palindrome(s[1:-1])
```

6. Write a function to find the GCD of two numbers using recursion. (Greatest Common Divisor (GCD))

```
def gcd(a, b):
    if b == 0:
        return a
    else:
        return gcd(b, a % b)
```

7. Write a function to generate all permutations of a string using recursion.

```
def permutations(s):
    if len(s) == 0:
        return ['']
    else:
        perm_list = []
        for i in range(len(s)):
            part = s[:i] + s[i+1:]
            for p in permutations(part):
                perm_list.append(s[i] + p)
        return perm_list
```

8. Write a function to find the sum of digits of a number using recursion.

```
def sum_of_digits(n):
```

```
if n == 0:
    return 0
else:
    return n % 10 + sum_of_digits(n // 10)
```

9. Write a function to count the number of vowels in a string using recursion.

```
def count_vowels(s):
    vowels = 'aeiouAEIOU'
    if not s:
        return 0
    else:
        return (1 if s[0] in vowels else 0) + count_vowels(s[1:])
```

10. Write a function to find the minimum element in a list using recursion.

```
def find_min(lst):
    if len(lst) == 1:
        return lst[0]
    else:
        return min(lst[0], find_min(lst[1:]))
```

11. Write a function to count the number of times a specific element appears in a list using recursion

```
def count_occurrences(lst, x):
    if not lst:
        return 0
    else:
        return (1 if lst[0] == x else 0) + count_occurrences(lst[1:], x)
```

12. Write a function to find the length of a list using recursion.

```
def list_length(lst):  
    if not lst:  
        return 0  
    else:  
        return 1 + list_length(lst[1:])
```

13. Write a function to count the number of digits in a number using recursion.

```
def count_digits(n):  
    if n == 0:  
        return 0  
    else:  
        return 1 + count_digits(n // 10)
```

14. Write a function to find the sum of even numbers in a list using recursion.

```
def sum_even_numbers(lst):  
    if not lst:  
        return 0  
    else:  
        return (lst[0] if lst[0] % 2 == 0 else 0) + sum_even_numbers(lst[1:])
```

15. Write a function to find the sum of squares of the first n natural numbers using recursion.

```
def sum_of_squares(n):  
    if n == 0:  
        return 0  
    else:  
        return n**2 + sum_of_squares(n - 1)
```

16. Write a function to count the number of consonants in a string using recursion.

```
def count_consonants(s):
    vowels = 'aeiouAEIOU'
    if not s:
        return 0
    return (1 if s[0].isalpha() and s[0] not in vowels else 0) + count_consonants(s[1:])
```

17. Write a function to calculate the product of digits of a number using recursion.

```
def product_of_digits(n):
    if n == 0:
        return 0
    if n < 10:
        return n
    return (n % 10) * product_of_digits(n // 10)
```

18. Write a function to count the number of uppercase letters in a string using recursion.

```
def count_uppercase(s):
    if not s:
        return 0
    return (1 if s[0].isupper() else 0) + count_uppercase(s[1:])
```

19. Write a function to check if a number is a prime using recursion.

```
def is_prime(n, divisor=None):
    if divisor is None:
        divisor = n - 1
    if n <= 1:
        return False
    if divisor == 1:
        return True
    if n % divisor == 0:
```

```
    return False
return is_prime(n, divisor - 1)
```

20. Write a function to check if a number is a palindrome using recursion

```
def is_palindrome_number(n, temp=None):
    if temp is None:
        temp = n
    if n == 0:
        return temp == 0
    return is_palindrome_number(n // 10, temp - (n % 10) * (10 ** (len(str(n)) - 1)))
```

21.